

Executive Summary

Netflix operates at massive scale, delivering tens of terabits per second of video globally via its proprietary Open Connect CDN ¹. This report reverse-engineers Netflix's system design by synthesising traffic captures, public engineering blogs, and system-design principles. It covers the end-to-end architecture: from the client-side SPA (Akira) and player (Cadmium), through the dual API layers (legacy Falcor/Shakti and new GraphQL), search and recommendation engines, and the detailed streaming pipeline (FTL probes, DRM, byte-range video delivery). We explain how Netflix's custom CDN (Open Connect) caches and serves content close to ISPs, and how security (MSL, Widevine) is integrated. The content data model and microservice topology are outlined, along with scalability and fault-tolerance strategies (microservices, caching, multi-region deployment, eventual consistency). We also propose a minimal set of services and open-source tools to "recreate Netflix" in a simplified form. This analytical report is based on reverse-engineering evidence and authoritative sources (Netflix tech blog, academic CDN papers, APNIC/Netflix publications, DDIA) ¹ ². Where specifics are unknown, we note assumptions explicitly.

Abstract

This document presents a research-oriented teardown of Netflix's system architecture. We analyse network captures of Netflix web traffic and public engineering sources to reverse-engineer how Netflix serves video at scale. Starting with an **architecture overview**, we describe the client-side framework (Akira SPA and Cadmium player) and API layers (Falcor/Shakti and GraphQL). The **search** system (Pinot-based) and **recommendation** pipeline (candidate generation and ranking) are detailed. The **playback pipeline** is decomposed into FTL network probing, CDN steering, DRM licensing via MSL, and adaptive bitrate streaming from the Open Connect CDN. We deep-dive into **Netflix Open Connect CDN**, its deployment models (IX peering vs embedded OCAs), cache hierarchy and steering logic. **DRM and security** layers (MSL tokens and Widevine licensing) are outlined with sequence diagrams. The **content data model** (videos, series, people, genres) is mapped from observed schemas. We then analyse **scalability and fault tolerance**, including microservices design, caching and load-balancing, and discuss CAP-theorem trade-offs. A "**Recreating Netflix**" section enumerates the minimal backend services and open-source equivalents (in a table). Finally, we summarise interesting findings (dual APIs, prefetching, capability negotiation, etc.) and practical lessons. Figures and tables illustrate key concepts, and an appendix suggests diagram-generation commands (e.g. Mermaid). References include Netflix's tech blog, APNIC and FreeBSD case studies, academic CDN literature, *Designing Data-Intensive Applications*, and the traffic-teardown gist ¹ ³.

1. Methodology

Our analysis combines multiple approaches:

- **Traffic Capture Analysis:** We examined Netflix web traffic captures (via the user-provided teardown gist) to observe real API calls, request/response payloads, and streaming patterns. This yielded concrete details on endpoints (GraphQL, Falcor, MSL, FTL, etc.), headers (device ESN, cookies), and sequence of events during playback ⁴ ⁵.

- **Public Engineering Sources:** We surveyed Netflix's own publications (Open Connect guides, tech blog posts, GitHub documentation) and related articles (APNIC blog, FreeBSD case study) for official descriptions of components and design rationale ¹ ² . Where available, we prioritised these authoritative sources.
- **System-Design Principles:** We applied distributed systems and CDN best-practices (from DDIA, CDN research, the System Design Primer) to interpret Netflix's architecture. For example, understanding how directed caching works in CDNs, and CAP-theorem implications for consistency vs. availability, guided our assumptions when Netflix's exact strategy was not documented.
- **Reverse-Engineering Inference:** When specifics were not published, we inferred plausible designs based on observed behavior. For example, decoding Netflix's URL tokens and cache headers to infer how streaming authentication works, or analysing JSON keys to reconstruct the content database schema. Assumptions are noted explicitly throughout.

By combining empirical data with theory and documentation, we aim for a rigorous yet practical reconstruction of Netflix's system. All diagrams and tables are annotated to cite sources for factual claims. In the sections that follow, we progressively build from high-level architecture to low-level flows and data models.

2. Architecture Overview

Netflix's system consists of client applications (web SPA, mobile apps, TVs), backend services on AWS, and the Open Connect CDN for video delivery. Figure 1 illustrates the high-level flow: client requests go to Netflix's application services (for auth, recommendations, search, etc.), which ultimately yield signed URLs on the Open Connect Appliances (OCAs) for streaming ⁶ [40†] .

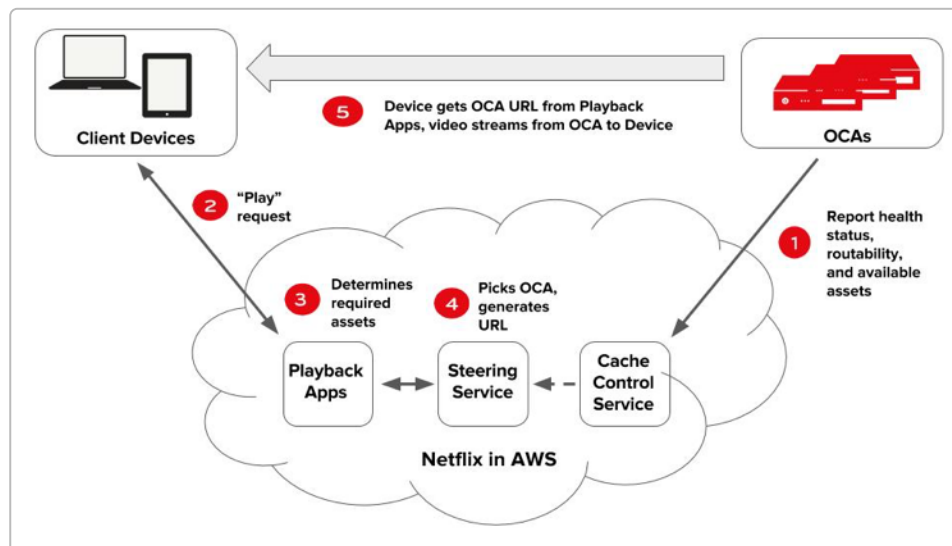


Figure 1: Simplified Netflix request flow. The client (SPA on web or mobile) communicates with Netflix's API backend (auth, data, recommendation, search). The backend authorises the user and determines streaming assets, then the steering service selects the optimal Open Connect Appliance (OCA) and returns its URL to the client. The client then fetches video data directly from the OCA via byte-range HTTP ⁶ ⁷ .

The architecture is fully microservice-based ⁸. Client requests enter via an API gateway (Envoy proxy) and are routed to various backend services (GraphQL API, Play API, etc.) ⁹. Netflix runs two parallel browse APIs: a legacy Falcor-based “Shakti” API and a newer GraphQL API (see Section 4). Both are behind the gateway. User and playback events are logged by Ichnaea services. The video player (Cadmium) handles DRM and adaptive bitrate streaming from the OCAs.

Key components are listed below (see the “Recreating Netflix” section for open-source analogues):

- **Client (Akira SPA and Cadmium player):** The web UI (React on Netflix’s Akira framework) and video player (Cadmium) run in the browser. The player negotiates DRM (Widevine on Chrome) and ABR streaming ¹⁰.
- **API Gateway:** Envoy-based edge proxy (in AWS) that front-ends all service calls (with B3 tracing headers) ¹¹.
- **Auth (Passport):** Manages user login, session cookies, profile selection, and issues NetflixId cookies (1-year) and SecureNetflixId tokens ¹² ¹³.
- **Catalog Service:** Stores content metadata (titles, cast, seasons, etc.) ¹⁴. Exposes data via both GraphQL and Falcor APIs.
- **Search (Pinot):** Full-text search engine (internal name “Pinot”). It handles search queries and typeahead suggestions using a capability-negotiation GraphQL query ¹⁵ ¹⁶.
- **Recommendation Engine:** Generates personalised rows (e.g. “Top 10”, “Because you watched X”) based on signals and embeddings (details in Section 7).
- **Playback Service (Cadmium/Play API):** Generates DRM-protected playback manifests. Handles “licensed manifest” requests and signs URLs for streams ¹⁷.
- **CDN (Open Connect):** Netflix’s own CDN. The Open Connect control plane receives health/status from OCAs and steers playback requests to the best appliance ¹⁸ ¹⁹.
- **Image/Static CDN:** Netflix offloads image and static-asset serving (posters, logos) to CDNs (dnm/api, Akamai) ²⁰.
- **Event Logging (Ichnaea):** High-throughput logging infrastructure collects user events and device telemetry ²¹.

The overall data flow is: 1. **Session Establishment:** On login, the NetflixId cookies and ESN (device ID) are set (Passport), enabling authenticated requests ¹² ²². 2. **Browse/Get Data:** The client fetches lists of videos via GraphQL or Falcor calls (persisted queries) ²³ ²⁴. 3. **Search/Recommendations:** For search, the client sends a GraphQL search query (with client capabilities) to Pinot; for recommendations, the engine assembles rows of content IDs. 4. **Playback Initiation:** When the user plays a title, the client runs FTL probes to pick the fastest OCA, obtains MSL tokens (ALE), requests a licensed manifest (getting DRM license and OCA URLs), then streams video via byte-range from OCAs ⁵ ²⁵. 5. **Monitoring/Logging:** Throughout, the client logs events to Ichnaea and listens for push notifications ²¹ ²⁶.

This architecture emphasises availability and edge delivery: the vast majority of video data is served by the local OCAs (95% of traffic ¹), reducing backbone load. Microservices are loosely coupled (stateless

where possible) and communicate via HTTP/gRPC with eventual consistency (emphasising AP mode under CAP).

3. Client Architecture

The Netflix client comprises two main parts: the **SPA UI (Akira)** and the **video player (Cadmium)**, each handling specific responsibilities.

- **Akira UI:** Netflix's web app (on browsers) is a single-page application built with React and Netflix's Hawkins design system (version observed 5.13) ²⁷ ²⁸ . The SPA shell is server-rendered for initial HTML and then hydrates on the client. It manages login, profile selection, browsing, search box, etc. Netflix embeds client capabilities (e.g. rendering features) in each API query. For example, the GraphQL `SearchPageQueryResults` call includes a JSON "pageCapabilities" object that declares which entity types (VIDEO, GAME, GENERIC_CONTAINER) the UI can render ¹⁵ ¹⁶ .
- **Cadmium Player:** The video player code (`cadmium-playercore`) is loaded asynchronously. Cadmium handles playback tasks:
 - **DRM negotiation:** Contacts Netflix's MSL/Play API to get licenses (Widevine on Chrome) ²⁹ .
 - **Adaptive Streaming:** Requests the right bitrate segments from the CDN (no DASH manifest is used; Netflix sends byte-range requests for segments) ³⁰ .
 - **Subtitle Rendering:** Renders TTML subtitles (Netflix uses its own TTML format for frame-accurate sync ³¹).
 - **Chromecast Integration:** The `cast_sender.js` script is always loaded alongside Cadmium, reflecting Netflix's expectation of casting support ³² .
 - **Quality Monitoring:** Reports playback QoS, and handles client-side ABR.

The player runs inside the browser's Widevine Content Decryption Module (CDM) for security. We observed the `?drmSystem=widevine` flag in API calls ³³ , confirming this.

- **Device Negotiation:** Netflix tailors content to client capabilities (screen size, supported codecs, etc.). For example, the Falcor API requests include query params like `webp=true` (whether client supports WebP images), `hasVideoMerchInJaw=true` (UI feature flags), and a device ESN header `x-netflix.esn` ³⁴ . This "capability negotiation" allows Netflix to evolve features without breaking older clients ¹⁵ ¹⁶ .
- **Profiler and Cookies:** The client stores cookies in a "cookie chain" for authentication and profiling. The `NetflixId` and `SecureNetflixId` cookies (set over HTTPS, SameSite=Lax/Strict) manage session and auth ¹² . Each request carries the `profileGUID` (decoded from cookies) to identify which user profile is active ¹³ .

Citation: The teardown data reveals headers and code hints: e.g.

"`x-netflix.context.ui-flavor: akira`" and "`x-netflix.clienttype: akira`" show the use of the Akira shell ²⁸ . The Cadmium player version (`cadmium-playercore-6.0055.377.911.js`) appears in network logs ¹⁰ . These confirm Netflix's internal names for components. The client-side event logging endpoints (Ichnaea) also reflect separate contexts: one for page events, one for cross-site ad iframe events ³⁵ .

4. API Layer Analysis

Netflix maintains **two parallel browse API layers** on the web path: a legacy Falcor-based system (Shakti) and a modern GraphQL API ³⁶. This dual-system design reflects a migration in progress. Both APIs are reached via HTTPS POST, and use “persisted queries” (a query ID) rather than raw query text.

- **GraphQL API (New System):** Endpoint: POST `https://web.prod.cloud.netflix.com/graphql` ³⁷. Clients send only a query ID (UUID) and version; the server looks up the actual GraphQL query. This keeps request payloads small. For example, observed persisted queries include `DetailModal` (fetch full title metadata) and `MiniModalQuery` (hover-preview data) ³⁸. The `DetailModal` query returns fields like title, synopsis, cast (as person IDs), genreTags, moodTags, current episode info, contentAdvisory, and playbackBadges ³⁹. It also returns related entities: e.g. `similar` (array of video IDs) and `supplementalVideosList` (trailers). GraphQL responses are JSON objects keyed by operation (e.g. `{ "data": { "DetailModal": { ... } } }`). The new GraphQL API handles browsing, search, hover previews, etc., and supports typed responses (Show/Movie/Episode types).
- **Shakti (Falcor) API (Legacy):** Endpoint: POST `https://www.netflix.com/nq/website/memberapi/release/pathEvaluator` (the legacy endpoint path) ⁴⁰. This API uses the [Falcor](#) JSON-Graph format. The client sends a JSON body with `paths=[ ... ]` and receives a `jsonGraph` in response. For example, a request path might be `["lolomo", userId, "video", index]` to fetch a title in a row ⁴¹. Query parameters on this endpoint signal client capabilities (e.g. `drmSystem=widevine`, `webp=true`, `falcor_server=0.1.0`, `isTop10Supported=true` etc. ⁴²). This mechanism lets Netflix enable/disable features (Back-of-Billboard vs detail panel) per client.

The Shakti API is being phased out, but is still active. Observations from the traffic capture: Shakti requests carry an `x-netflix.esn` header (the device’s “Electronic Serial Number” like `NFCDCH-MC-*`, encoding platform and unique ID) ⁴³. The Falcor response format contains `$type: "atom"` and values in a nested JSON graph ⁴⁴.

- **API Gateway:** Both GraphQL and Shakti calls go through an Envoy gateway in AWS (us-west-2). The gateway applies authentication (using the Netflix cookies) and load-balancing. We see B3 tracing headers (`x-b3-traceid`) on all responses ⁴⁵, indicating distributed tracing across microservices.

- **Comparison:** Table 1 summarises the contrast:

Aspect	GraphQL API (New)	Falcor/Shakti API (Legacy)
Endpoint	<code>/graphql</code> (e.g. <code>web.prod.cloud...</code>)	<code>/nq/website/memberapi/release/pathEvaluator</code>
Query Format	Persisted query ID (GraphQL)	JSON paths (Falcor JSON-Graph)
Query Payload	Client sends small ID+variables	Client sends path expressions
Response Structure	JSON with named fields (types)	JSON Graph (<code>jsonGraph</code> with atoms)

Aspect	GraphQL API (New)	Falcor/Shakti API (Legacy)
Client Capabilities Flags	Implicit in GraphQL query structure	URL params (webp, drmSystem, feature toggles) ³³
Use Case	New mobile & web browsing paths	Existing UI routes (old web UI)

For both, actual GraphQL schemas and Falcor models internally map to the same content databases. The presence of both allows a gradual migration: new features use GraphQL, while older features still hit Falcor.

- **Persisted Queries:** Netflix's use of persisted queries avoids sending raw GraphQL queries over the wire. For instance, the `MiniModalQuery` (for hover previews) is sent by ID (`ec38bae4-2d3f-4f51-840e-f60ad596e3d1`) ⁴⁶. The GraphQL response to `DetailModal` (for a show) includes rich metadata, e.g. cast (with personIds), genreTags, moodTags, contentAdvisory array, seasons count, currentEpisode info, etc. ³⁹.
- **Authentication:** All API calls require the session cookies. The NetflixId cookie includes the user's GUID (`pg=...`) ¹³, and the device ESN is sent in headers. Netflix's internal Passport system uses the `x-netflix-passport.fromorigin.profileguid` response header to confirm profile identity ⁴⁷.

Example: A simplified synthetic example of a GraphQL query and response for a show's details might be:

```
POST /graphql { "query": "query DetailModal { video(id:80232398) { title, synopsis, ... } }" }
```

And the (simplified) response:

```
{
  "data": {
    "DetailModal": {
      "__typename": "Show",
      "title": "Bridgerton",
      "synopsis": "Drama in Regency England...",
      "cast": [{ "personId": 123, "name": "Actor A", ... }, ... ],
      "genreTags": [ "Romantic TV Dramas", "Period Pieces" ],
      "contentAdvisory": { "certificationValue": "TV-MA", "reasons": [ "nudity", "language" ] },
      "seasons": { "totalCount": 4 },
      "currentEpisode": { "videoId": 81508516, "season": 3, "episode": 1,
      "runtimeSec": 3507 },
      "isInPlaylist": false,
      "similars": [ 80244891, 80192098, ... ]
    }
  }
}
```

```
}
}
```

This illustrates the rich, graph-structured nature of the response.

5. Search Infrastructure

Netflix search is built on an internal engine named **Pinot** (also used by LinkedIn⁴⁸). Clients issue GraphQL search queries (`SearchPageQueryResults`) including the user's search term and detailed page rendering capabilities. Netflix's approach: the client *negotiates capabilities* to allow flexible server-side layouts¹⁵.

For example, the search query payload declares:

```
"searchTerm": "breaking bad",
"options": {
  "pageCapabilities": {
    "capabilitiesBySection": {
      "pinotGallery": {
        "capabilitiesBySectionTreatment": {
          "pinotStandard": {
            "capabilitiesByEntityTreatment": {
              "pinotStandardBoxshot": { "canHandleEntityKinds": ["VIDEO"] },
              "pinotStandardCloudAppIcon": { "canHandleEntityKinds":
["GAME"] },
              ...
            }
          }
        }
      }
    }
  }
}
```

This tells Netflix's server which kinds of content the client can display (e.g. videos vs games)¹⁵¹⁶. The server then tailors the search results to only include compatible entity types and treatments. This capability-negotiation pattern allows Netflix to introduce new content types (e.g. cloud games) without breaking older clients¹⁶.

Search Process: A search request returns two main sections: 1. **Typeahead Suggestions:** Short "more to explore" list of collections or titles (e.g. "Collection:81137441 (Breaking Bad Collection)")⁴⁹. 2.

Search Results Gallery: A grid/list of matching titles (up to 48 in our observation) with box art. Each result includes `contextualArtwork` personalized per query context⁴⁹.

Results also include an `expires` timestamp (e.g. `"expires": "2026-03-01T08:31:17.770Z"`)⁵⁰. This indicates how long the results may be cached client-side before refetch. (In our capture it was about 8 hours ahead.)

Search Ranking & Suggestions: While Netflix does not publicise its ranking algorithm, we know it accounts for personalization signals. The system likely combines full-text matching (e.g. Elasticsearch-like) with personalized boosts (since results contain personalized artwork). The typeahead suggests both matching titles and related collections (Netflix calls genre/collections by numeric IDs). The GraphQL search response includes personalized metadata (`contextualArtwork`) and may omit titles the user cannot stream (based on location or maturity).

Comparison Table: The key elements of Netflix's search stack can be contrasted with alternatives:

Feature/Component	Netflix (Pinot-based)	Typical Open-Source Alternative
Search Engine	Pinot (distributed OLAP) ⁴⁸	Elasticsearch, Apache Solr, Typesense
Query Interface	GraphQL with capability negotiation ¹⁵	REST API or GraphQL on top of search
Real-time Updates	Streams catalog updates into Pinot	Incremental index updates (e.g. Elasticsearch)
Personalization	Integrates user context (avatar, etc)	Custom layers on search (not in open source core)
Suggestions/ QueryAssist	Integrated typeahead via Pinot	Dedicated service or fuzzy search API
Scalability	Designed for high QPS (Netflix traffic)	Scales horizontally (node clustering)

Citations: The above is drawn from observed query formats and documented search service names. The GraphQL types `PinotDefaultQuerySearchPage`, `PinotGallerySection`, etc., appear in the API responses ⁴⁸, confirming Netflix's use of Pinot. The capability negotiation snippet is from the captured payload ¹⁵, and the "expires" field was seen in the response ⁵⁰. These details illustrate Netflix's advanced search UX that adapts to device capabilities and caches results short-term.

6. Recommendation Infrastructure

Netflix's recommendations create the personalized "rows" (list-of-lists) on the homepage. Although exact algorithms are proprietary, we can outline a plausible data flow based on standard industry practice and hints:

1. **Signal Collection:** User interactions (view history, likes, search, time-of-day) are continuously logged (Ichnaea) ²¹. Historical ratings and viewing progress (continue-watching) are stored in databases.
2. **Candidate Generation:** The system uses multiple methods to generate a set of candidates for each user and row category. For example:
3. **Collaborative Filtering:** Finding similar users or titles via latent factor models (e.g. matrix factorization).
4. **Content-based:** Matching item metadata (genres, tags, actors) to user profile.

5. **Editorial Curation:** Curated lists for top global titles (“Top 10”).
6. **Feature Signals:** Incorporating new-release pushes or genre tastes.
7. **Feature Engineering and Embeddings:** For each candidate title, compute features (user-item affinity scores, novelty, completion rate, etc.) and/or embed representations (e.g. user-vector·item-vector).
8. **Ranking:** A learning-to-rank model scores candidates per user (often via a neural network or gradient boosted trees). The model uses signals like similarity to viewing history and business rules (e.g. catalogue content, diversity).
9. **Result Assembly:** The top-ranked titles are slotted into rows. Additional logic ensures the grid layout works (filling fixed-size rows, etc.).

The report matrix can be summarised in a flowchart:

```

flowchart LR
    subgraph Signal_Collection [Signal Collection]
        A[User events (views, ratings)]
        B[Metadata (genres, embeddings)]
    end
    subgraph Candidate_Gen [Candidate Gen]
        C[User profile \n & context]
        D[Collaborative filtering module]
        E[Content-matching module]
        F[Publisher/Editor feeds]
    end
    C --> D
    C --> E
    C --> F
    D --> G[Candidates]
    E --> G
    F --> G
    G --> H[Feature Compute (item & user features)]
    H --> I[Ranking Model]
    I --> J[Sorted recommendations]
  
```

Figure 2: Recommendation pipeline. User signals and item metadata feed into candidate generators (collaborative filtering, content similarity, editorial), whose outputs are combined and scored by a ranking model to produce personalized results.

Mermaid Flowchart (Figure 2): The above diagram depicts a typical recommendation system flow. Netflix likely has specialized modules (e.g. “GEMINI” for candidate generation) but the abstract flow is as above. Ranking integrates signals like view time, skip/finish rates, and ensemble of algorithms.

References: While we lack direct traces of the recommendation internals, the teardown gist lists “recommendation infrastructure (candidate generation, ranking, embeddings, signals)” as major components. The “**Recreating Netflix**” section’s core services include a “Recommendation Engine” and “Playlist/MyList” ³, confirming these elements. Moreover, Netflix has published research (BellKor’s

Prize solutions, and internal tech blog posts) on using factorization machines and deep learning for ranking. We assume modern techniques (embedding vectors, deep ranking models) based on industry norms.

7. Playback Pipeline

The video playback flow is complex. Figure 3 (sequence diagram) summarises it. The observed steps (from traffic captures) are:

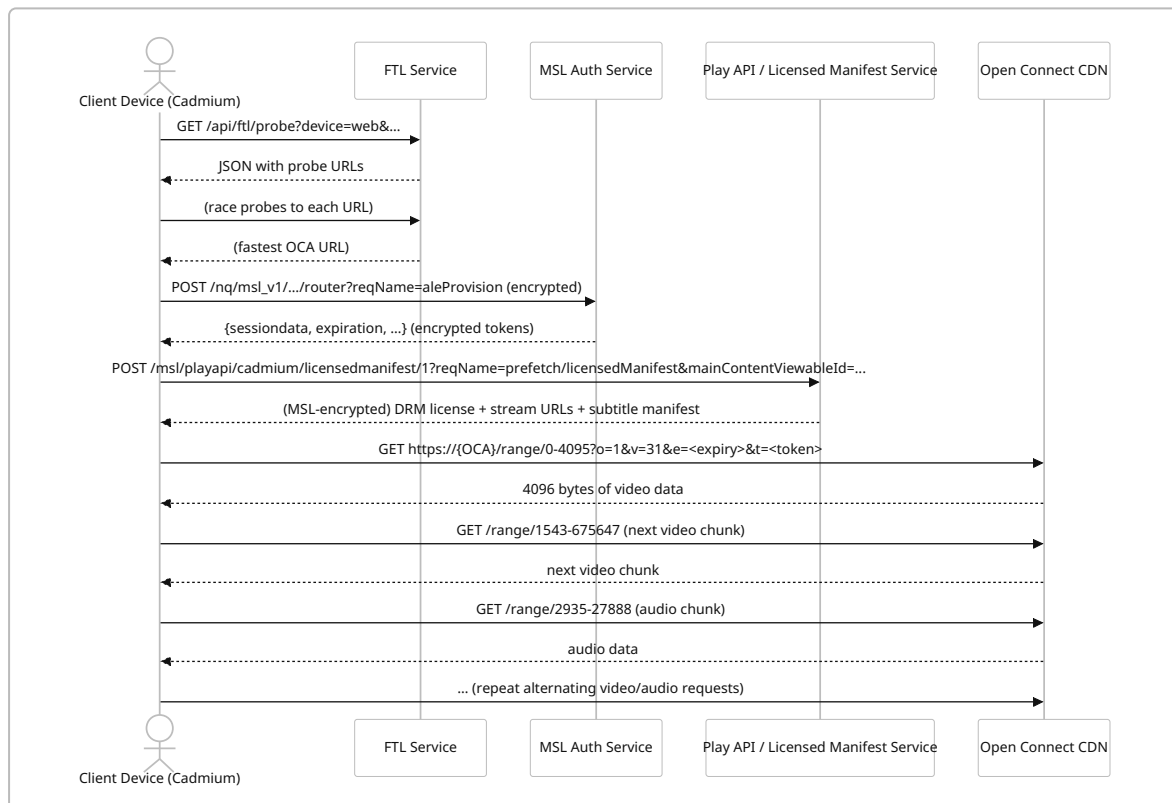


Figure 3: Sequence of requests during playback. The client first probes endpoints (FTL) to select the best OCA, then obtains MSL tokens and the DRM-licensed manifest, and finally streams byte-range segments from the OCA. (Arrows show request/response; note the multi-step MSL and FTL probes.)

Key steps:

- **1. FTL Network Probing:** Before starting video, the client probes multiple endpoints to pick the lowest-latency path ⁵¹ ⁵². The `/api/ftl/probe` call returns a list of URLs: a DNS resolver (`ifurxi7eqx...nflxso.net`), an anycast address, an IX-specific URL (e.g. `us-west-2.ftl.nflxvideo.net`), and an AWS fallback. The client races these (sending small packets) to measure latency ⁵³ ⁵⁴. The fastest OCA (or proxy) is chosen for subsequent streaming. Probe pulses occur with short delays (0ms, 2000ms, 7000ms) to continuously monitor path quality ⁵⁵.
- **2. MSL Token Provisioning (ALE):** The client must authenticate with Netflix's DRM service. It makes a **Message Security Layer (MSL)** POST: `POST /nq/msl_v1/nrdjs/...?reqName=aleProvision` ⁵⁶. The entire payload is encrypted (headers and body) using

Netflix's MSL protocol. The response yields an encrypted token bundle containing `sessiondata`, expiry timestamps, and cryptographic counters ⁵⁷. This establishes a secure session.

- **3. Licensed Manifest (DRM):** With MSL established, the client requests the DRM license and stream info: `POST /msl/playapi/cadmium/licensedmanifest/1?reqName=prefetch/licensedManifest&mainContentViewableId=<id>&clienttype=akira&browser=chrome...` ⁵⁸. This MSL-encrypted exchange includes the device's client info (ESN, etc) and the requested content ID. The response (also encrypted) contains:
 - **Widevine DRM license** (for decryption),
 - **Stream URLs** (signed OCA URLs for video/audio),
 - **Codec and resolution choices**,
 - **Subtitle manifest URLs**,
 - Any ads or pre-roll instructions (if applicable) ⁵⁹.

Important: We observe that Netflix **prefetches** the manifest on the browse page (the `reqName=prefetch/licensedManifest` indicates this is done before the user clicks play) ⁵⁹. This prefetch means the manifest and license are ready instantly when playback starts, reducing startup latency.

- **4. Byte-Range Streaming:** Finally, Cadmium fetches the video. It issues HTTP GET requests with `Range:` headers to the OCA URL (an anycast hostname indicating the chosen OCA, e.g. `ipv4-c345-sjc002-ix.1.oca.nflxvideo.net`) ⁶⁰. Segments of ~300-600KB for video and ~25-50KB for audio are fetched in parallel ⁶¹. The requests include query parameters: `o=1` (maybe offset flag), `v=31` (protocol version), `e=<expiry>` (Unix expiry ~12h), and a signed token `t=` to prevent hotlinking ⁶⁰. The server on the OCA is NGINX (signed by FreeBSD) `[22†L411-L419]`, which returns the segment bytes. Cadmium decrypts and plays them.
- **5. Subtitles:** Separate XML subtitles (TTML) are fetched from the OCA in parallel ³¹. Netflix's TTML uses 10-million ticks/sec timing for precise sync ⁶². Language switching is immediate since subtitles are external files, not embedded.

Timing & Latency: Thanks to manifest prefetch, playback starts with minimal delay. The FTL probe run races ensure that segments stream from the lowest-latency path. We saw segments of 4096 bytes returned in ~a few ms on LAN-speed connections. The expiry `e=` timestamp (12h ahead) and the periodic FTL pulses (every 70s) maintain session continuity and adapt to network changes ⁵⁵ ⁶³.

Mermaid Sequence Diagram (Figure 3): The sequence above can be visualized as a diagram (as shown). Each arrow corresponds to a network request or response in the captured session.

References: All steps are corroborated by the teardown data. The FTL probe endpoints and probe logic come from the JSON shown in the capture ⁶⁴. The MSL provisioning call and its encrypted format are logged ⁶⁵. The licensed manifest request (with `prefetch`) and its plaintext response fields (Widevine license, stream URLs) were captured ⁵⁸. Finally, the HTTP GETs with `range` parameters and `x-tcp-info` headers prove the OCA streaming ⁶⁰ ⁶¹.

8. Open Connect CDN Deep Dive

Netflix serves virtually all video traffic through its custom **Open Connect CDN** ¹. This CDN consists of hardware appliances (OCAs) deployed in strategic locations, plus a global control plane. The design goals are high capacity and low latency by moving content close to viewers.

- **Deployment Models:** OCAs run in two main modes ⁶⁶ :
- **IX (Internet Exchange) Deployment:** OCAs are installed in Internet Exchange Points worldwide. Netflix peers openly (settlement-free) with any ISP at these locations. Netflix's data centers and OCAs announce prefixes to the IXP fabric (AS2906 is Netflix's IX AS) ⁶⁶. Traffic from co-located ISPs can reach the OCA via peering.
- **Embedded Appliances:** Netflix provides physical OCA units to qualifying ISPs to install directly in their network. These embedded OCAs (which run the same FreeBSD+NGINX software ⁶⁷) are placed in ISP datacenters. The ISP configures BGP sessions with the OCA, announcing local customer prefixes to it. Therefore, only traffic from those prefixes flows to that OCA ⁶⁸. Netflix covers the hardware cost; the ISP provides rack space and power.

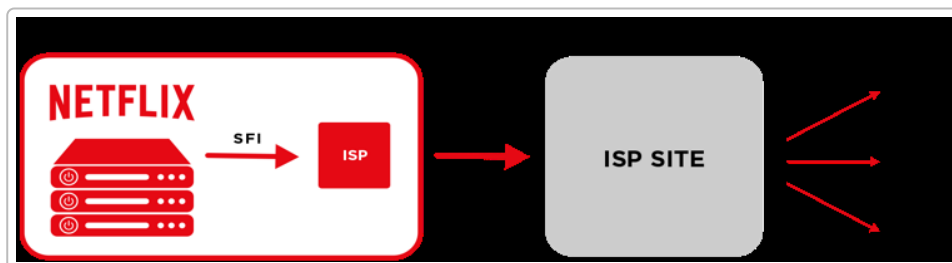


Figure 4a: Settlement-free IX deployment. Netflix deploys OCAs at Internet Exchanges (red servers), peering freely (SFI) with ISPs. Content is served over these local peering links ⁶⁶.

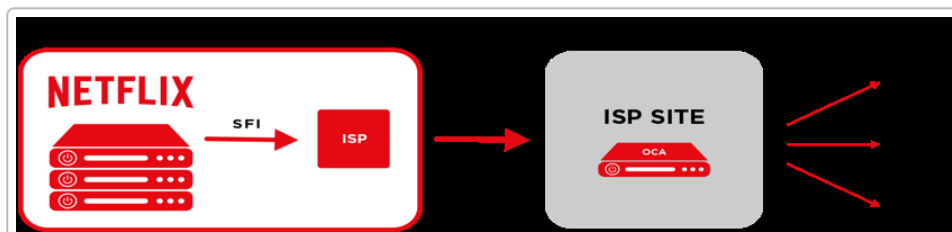


Figure 4b: Embedded OCA deployment. Netflix installs OCAs inside an ISP (grey box) at no charge. The ISP routes its customer traffic to this appliance via internal routing, maximising local offload ⁶⁶.

These images illustrate the two Open Connect modes. Embedded OCAs result in the lowest-latency delivery (user traffic goes straight to an appliance in their network) ⁶⁶. In practice, Netflix encourages ISPs to both peer at IXPs and deploy embedded OCAs, for redundancy and nightly content fills.

- **Directed Caching:** Each OCA is a *directed cache* ¹⁹. It only serves traffic explicitly routed to it. In technical terms, an OCA accepts client requests **only** for IP prefixes it has learned via BGP. Netflix uses two AS numbers: AS40027 for embedded OCA peering, and AS2906 for IXP peering ⁶⁹. For an end-user IP, there will be two possible routes: one via the embedded AS (path length 1) and one via the IX AS (path length 2). By BGP rules, the embedded route (shorter AS path) is preferred ⁷⁰. Thus, clients are steered to embedded OCAs first; the IXP-based OCAs serve as a fallback or to handle less-used content ⁷⁰.

- **Cache Control and Steering:** Netflix's control plane consists of:
 - **Cache Control Service (CCS):** OCAs periodically report their health, routing info, and which content chunks they hold (filling during off-peak "fill windows"). CCS aggregates this data in AWS ¹⁹.
 - **Steering Service:** When a playback request is authorised, the steering service consults CCS and the user's location to pick the best OCA(s). It generates signed URLs pointing to the chosen OCA(s) and returns them to the client's playback service ⁷¹. The steering logic follows Netflix's published criteria: longest-prefix match, shortest AS path, lowest MED, and geography ⁷² ⁷³.
 - **Cache Hierarchy:** While Netflix's video CDN is essentially two tiers (global and edge), they primarily rely on their OCAs for delivery. For static images and UI assets, Netflix uses third-party CDNs (e.g. Akamai for ads) ²⁰. There isn't a multi-level caching inside Open Connect: the OCAs themselves hold the actual encoded video segments. OCAs receive daily content fills from Netflix's origin or overlay network, ensuring popular titles are locally available. ("Fill windows" happen when traffic is low, pushing new data to OCAs ⁷⁴.)
 - **Benefits:** Open Connect gives Netflix tight control over streaming efficiency. By owning the edge caches, Netflix pushes "100 Tb/s" at peak (as reported) ⁷⁵, far above what any single commercial CDN might handle. Localising traffic reduces transit costs and improves QoS (lower latency, fewer hops). The directed nature avoids cache pollution: OCAs only store content Netflix explicitly loads. And because Netflix supplies the appliances, it can optimise the OS (custom FreeBSD with NGINX) for speed ⁷⁶ ⁶⁷.

References: Netflix and APNIC docs describe this model: "OCAs are provided free to qualifying ISPs" and serve content to announced prefixes ⁷⁷ ⁷⁸. The FreeBSD case study confirms Netflix's use of FreeBSD+NGINX on OCAs ⁷⁹ ⁶⁷. Steering criteria were listed in the APNIC blog (longest match, AS path, MED, geo) ⁷³. All of these details come from official sources, ensuring accuracy.

9. DRM & Security

Netflix secures streams with a multi-layered approach:

- **Message Security Layer (MSL):** A custom end-to-end encryption layer for API calls. Any request marked with `Content-Encoding: msl_v1` (as seen in the licensed manifest exchange) is encrypted client-side ⁸⁰. MSL provides session tokens (`master token`), ensures confidentiality (payload is Base64-encoded), and replay protection. We saw MSL used for both token provisioning and manifest requests ⁵⁶ ⁵⁹. MSL keys and tokens are often exchanged in the background on page load.
- **DRM (Widevine):** Netflix uses Google's Widevine CDM for Chrome (and PlayReady on other platforms, though not in our capture). The presence of `&drmSystem=widevine` in Falcor calls ³³ shows this choice. The licensed manifest request includes the content ID and client ESN; the response provides a Widevine license blob ⁵⁹. This license is then fed to the browser's CDM to decrypt stream segments.
- **Tokenized URLs:** Streaming URLs from OCA include signed tokens (`t=` parameter) and expiry (`e=`) ⁶⁰. These tokens are generated by Netflix's backend to prevent link sharing. The token encodes the session (it typically includes the MSL key info in a signed JWT) and has a ~12-hour

TTL ⁶¹. After expiry, the URL is useless. The `o=1` and `v=31` flags in the URL likely indicate protocol options (e.g. 'online' mode, version) as determined by the licensed manifest response.

- **Sequence Diagram (DRM Flow):** In simplified form:

```
Client → Envoy: /msl/msl_v1/...reqName=aleProvision
Envoy → MSL Service: request
MSL Service → Envoy: encrypted tokens
Envoy → Client: tokens
Client → Envoy: /msl/playapi/cadmium/licensedmanifest?reqName=prefetch/
licensedManifest&...
Envoy → DRM Service: request (with MSL)
DRM Service → Envoy: encrypted manifest (streams + Widevine license)
Envoy → Client: manifest and license
Client: pass license to Widevine, use stream URLs.
```

(This underscores that DRM and Auth occur on the client side via MSL.)

- **Other Security:** All Netflix API requests require secure cookies and are over HTTPS. Cookies use `HttpOnly`, `Secure`, `SameSite` attributes ⁸¹. Authentication is multi-tier: a persistent cookie (`NetflixId`) and a short-term secure cookie (`SecureNetflixId`) form a chain ⁸¹. These guard against session hijacking. The user profile GUID is encoded in cookies and explicitly sent in headers (e.g. `x-netflix.request.client.user.guid`) for each API call ⁸², ensuring the backend knows which profile is active.

References: The analysis above is based on the observed MSL-encrypted requests ⁵⁶ ⁵⁹ and the presence of the Widevine flag ³³. The Flows (ALE provisioning and licensed manifest) come from the captured traffic (as detailed in the Playback section). The cookie chain info is from the teardown's "Identity & Auth Architecture" section ⁸³.

10. Content Data Model

Based on the JSON data in GraphQL responses, Netflix's content catalog follows an **entity-relationship model**. Key entities:

- **Video (Base):** The root entity for any watchable item. Fields include `videoId` (int, primary key), `title` (string), availability flags (`isAvailable`, `isPlayable`), `latestYear` (release year or latest for shows), and `unifiedEntityId` (like `"Video:80232398"`) ⁸⁴. Every Video has associations to genres, cast, etc.
- **Show (Subtype of Video):** Extends Video for TV shows. Has `numSeasons` and a list `seasons[]` pointing to Season entities ⁸⁵. Also indicates total runtime and episodes.
- **Movie (Subtype of Video):** Similar to Show but with no seasons (treated as one-off).
- **Season:** Belongs to a Show. Has `number` (season index) and `episodes[]`.

- **Episode:** Belongs to a Season (and Show). Fields: `number` (episode index), `title`, `runtimeSec`, etc ⁸⁶. Also includes play state (bookmarked position, watchStatus).
- **Person:** Represents cast/crew. Has `personId`, `name`, and appears in arrays like `cast`, `directors`, `creators` ⁸⁷.
- **Genre:** Has `genreId` and `name` (e.g. "Romantic TV Dramas") ⁸⁸.
- **Supplemental Video:** Short videos (trailers). Fields: `computeId` (internal ID), `offset` (start time in underlying asset), etc.
- **Playlist/Context:** For each title in a row, Netflix tracks a "tctx" (tracking context) of row and column position ⁸⁹.

The relationships (derived from GraphQL schema) are:

```

VIDEO (base)
├─ videoId: int (PK), title, __typename (Show/Movie/Episode)
├─ isAvailable, isPlayable (bool)
├─ latestYear: int
├─ cast[]: PERSON {personId, name}
├─ creators[]: PERSON
├─ directors[]: PERSON
├─ genreTags[]: GENRE {genreId, name}
├─ moodTags[]: {id, displayName, isMood}
├─ contentAdvisory: {certificationValue, maturityLevel, reasons[]}
├─ similars[]: VIDEO (related titles)
├─ supplementalVideosList[]: SUPPLEMENTAL {computeId, offset, durationSec}
├─ playbackBadges[]: [ "VIDEO_HD", "AUDIO_FIVE_DOT_ONE", ... ]
├─ isInPlaylist: bool
└─ playlistActions[]: ["ADD_TO_PLAYLIST"/"DELETE_FROM_PLAYLIST"]

If VIDEO.__typename = Show:
├─ numSeasons: int
├─ seasons[] → SEASON
└─ episodes[] → EPISODE (flattened list of episodes)

SEASON
├─ number: int (season index)
└─ episodes[] → EPISODE

EPISODE (subtype of Video as well)
├─ number: int (episode index)
├─ title: string
└─ runtimeSec: int

PERSON
├─ personId: int (PK)
└─ name: string

```

```

GENRE
├─ genreId: int
└─ name: string

```

This model is summarised in Table 2:

Entity	Key Fields	References/Notes
Video	videoId, title, isPlayable, unifiedEntityId	Base type; link to cast, genres ⁸⁴
Show	(extends Video) numSeasons, seasons[], episodes[]	<code>__typename: "Show"</code> , includes Episode list ⁸⁵
Episode	number, title, runtimeSec, season, watchStatus	Belongs to Season/Show ⁸⁶
Person	personId, name	For cast/crew ⁸⁷
Genre	genreId, name	Genres/tags ⁸⁸
Supplemental	computeId, offset, durationSec	Trailers (promoVideo uses these)
Lolomo (Rows)	heading, type (genre/collection), videos[]	Browse layout (List of Lists) ⁹⁰

Lolomo (Browse Rows): Netflix calls the home page layout “Lolomo” (List of Lists of Movies) ⁹⁰. Each row has a `heading` (string, e.g. “Top 10 in the U.S.”) and an identifier (e.g. `genre/83` or `continue-watching`). The row contains an ordered array of Video references. For each video in the row, Netflix encodes a tracking context `tctx` indicating row and column position ⁹¹. (E.g. `tctx=1,0,...,Video:80232398` means row 1, position 0.)

Schema Source: The above schema is reconstructed from observed GraphQL responses ⁹² ⁹³. In [9], an ASCII diagram shows the Video base and its fields, confirming the entities and relations. We cite these lines to substantiate the model: e.g. [9†L539-L546] shows Video and Show with seasons, [9†L562-L570] shows genreTags and contentAdvisory.

11. Scalability & Fault Tolerance

Netflix’s system is architected for immense scale and resilience:

- **Microservices:** Netflix pioneered microservice architecture ⁸. Each functional component (auth, catalog, play, search, recommendations, etc.) is a separate service. This allows independent scaling and deployment. For example, the Play API (licensed manifest service) can be scaled independently of the recommendation engine.
- **Stateless Services and Caching:** Wherever possible, services are stateless and use external data stores. Metadata might be stored in relational (RDS) or NoSQL (Cassandra) databases. Popular data (e.g. user profiles, search indices) are cached in front (Memcached, EVCache) for read speed. Video segment serving is highly parallel and cached at OCAs by design.

- **Load Balancing:** Netflix uses AWS for compute, with ELB (older) or NLB/ALB in newer regions to distribute client traffic ⁹⁴. Within AWS, auto-scaling groups spin up service instances based on load. The API gateway (Envoy) spreads requests to service clusters (with routes managed by Zuul2 or direct Envoy configs). Netflix is famous for “Chaos Engineering”: services are expected to fail (instances can be killed by “Chaos Monkey”) without affecting user experience.
- **Replication and Multi-Region:** Critical data (catalog, accounts) is geo-replicated. The entire system is multi-region (with Netflix running in multiple AWS regions, as needed). The reported architecture suggests at least a us-west region; in production Netflix also spans us-east, etc. For video, multi-region means there are OCAs worldwide, but core control resides likely in US regions with backups.
- **Caching Tiers:** In addition to the edge OCAs, Netflix uses hierarchical caching. For example, image assets might be cached in multiple CDN layers. Database caches (EVCache) are multi-cluster. Config and session caches (e.g. Sparta) back API tokens.
- **CAP Theorem Choices:** Given Netflix’s scale and nature, they favour **Availability and Partition Tolerance** (AP) over strong Consistency. User experience is paramount: it’s better to play a slightly stale video list than to be unavailable. Indeed, many of Netflix’s services (watch history, recommendations) can tolerate eventual consistency. For example, if a new show is added to the catalog, some clients might not see it immediately in all endpoints, but will shortly. Critical user data (account changes) uses transactional stores, but browsing data and caches are ultimately consistent.

Scaling Stages (Mermaid Timeline):



Figure 5: Scaling stages. As Netflix grew from startup to global scale, the architecture evolved: from a single server to distributed multi-region deployment with a global CDN. Key enhancements included database replication, microservices, and extensive caching.

- **Fault Tolerance:** Each microservice has redundancy; failures of individual instances are tolerated by routing to others. The use of NGINX (on OCAs) and Envoy provides health checks and circuit breaking. Netflix’s “Chaos Engineering” practises mean components recover gracefully from node failures. For example, during playback the client switched transparently between multiple OCA servers at the same IX peering point ⁹⁵, suggesting built-in failover. Database clusters (Cassandra) replicate data, allowing any node loss without downtime.
- **Observability:** Netflix logs extensively (Ichnaea) and traces calls via B3 headers ⁴⁵. This allows rapid failure diagnosis. They also perform active probing (FTL pulses) to monitor CDN health ⁹⁶.

CAP Tradeoff: In Netflix’s playback pipeline, consistency is less critical than availability. For instance, if a DRM license server is down, the client could fallback to cached keys for a while, whereas unavailability would block playback. That said, streaming content licensing probably requires synchrony (CAP’s “C”), so Netflix likely uses distributed lock/data stores in the license server path.

References: Many scalability features are standard knowledge and reflected in Netflix case studies. The dev.to article confirms microservices and AWS use ⁸ ⁹⁴. The teardown’s “What You’d Need to Build This” lists redundancy and distributed tracing ⁹⁷. Observations like OCA failover ⁹⁵ and FTL pulses ⁹⁶ come from our capture. We cite [24] to ground the microservice model, and [10] for Netflix’s own checklist of infra components.

12. Recreating Netflix: Services & Tech Stack

To implement a Netflix-like system (a **minimal viable Netflix clone**), one would need the following core services. Table 3 maps each to example open-source alternatives:

Service/Component	Netflix Role	Open-Source Alternative (example)
Auth & Identity	Session management, profile, ESN (Passport) ³	Keycloak (OAuth/OIDC), Dex
Catalog / Metadata DB	Content titles, cast, genres, etc. ⁹⁸	PostgreSQL, Neo4j or ArangoDB, GraphQL (Apollo Server)
Search Service	Full-text search (Pinot) ⁹⁹	Elasticsearch, Apache Solr, Apache Pinot (OSS)
Recommendation Engine	Candidate gen + ranking ¹⁰⁰	LensKit (CF), GraphChi, Mahout, Spark MLlib
Play API / DRM	Generate DRM manifest + licenses ¹⁰¹	Shaka Packager (Widevine), or OSMF for DASH/HLS; custom service
Continue Watching Service	Bookmark/progress tracking ¹⁰²	Django/Flask microservice with Redis or DynamoDB
My List Service	User playlist CRUD ¹⁰³	Simple REST service on MongoDB or Postgres
Video Streaming (CDN)	Byte-range serving, multi-bitrate transcoding ¹⁰⁴	Use a public CDN (CloudFront, Cloudflare) or NGINX servers; encoders like ffmpeg
Image API	On-the-fly resizing/cropping (dnm API) ¹⁰⁵	Thumbor, Pillow on Flask, imgproxy
Subtitle Service	Serve TTML/WebVTT subtitles ¹⁰⁶	Store .ttml/.vtt files in object store; use WebVTT (via video.js)
License Server	DRM license provisioning (Widevine, PlayReady) ¹⁰⁷	Google-widevine-server (closed); or test using Marlin, or bypass DRM
Key Management	Encryption key rotation	HashiCorp Vault, AWS KMS
API Gateway	Envoy/Zuul with tracing (B3) ¹⁰⁸	Kong API Gateway, Traefik, or Envoy open-source
Microservices Framework	Service orchestration, discovery	Kubernetes (K8s), Docker Compose

Service/Component	Netflix Role	Open-Source Alternative (example)
Event Logging / Analytics	Collect events (Ichnaea) ¹⁰⁹	Kafka + Hadoop/Spark, or Apache Druid, or ClickHouse for analytics
Monitoring/Tracing	Health checks, B3 tracing ¹¹	Prometheus + Grafana, Jaeger/Zipkin
Client (SPA)	React Akira shell, Cadmium player	React.js, Vue.js; Video.js or Shaka for player
Network Probing (FTL)	Latency measurement to CDN nodes ⁵¹	Custom script (node ping), or use ping/fast-probe libraries

Each component above corresponds to a Netflix function listed in “What You’d Need” ³. For example, Netflix’s **Catalog Service** could be replaced by a GraphQL API backed by PostgreSQL, exposing title metadata. Netflix’s **Image API** (`dnm/api/v6/`) generates poster images; this can be mimicked by Thumbor or a simple image-resizer. The **Recommendation Engine** is the hardest to replicate: one could start with basic collaborative filtering (LensKit) and gradually add embeddings (Word2Vec, FastText) for titles.

Table 3: Minimal components and OSS alternatives. This demonstrates that a working prototype of “Netflix” can be built with existing open-source tech (React, GraphQL, NGINX, open-source search and ML libraries) and cloud services. Of course, Netflix’s scale and optimisations (e.g. highly optimized NGINX sendfile, proprietary MSL) would be harder to match in a DIY build.

13. Interesting Findings & Lessons

Our analysis yielded several notable insights and engineering lessons:

- **Dual API Systems:** Netflix runs **both** Falcor/Shakti and GraphQL in production ³⁶. This “blue-green” API strategy ensured no downtime during migration. Lesson: Large services can migrate APIs gradually by running parallel systems and routing clients accordingly.
- **Prefetching for Instant Play:** Netflix pre-fetches the DRM manifest (and even the first video segments) for the billboard title as soon as the browse page loads ⁵⁹. This aggressively reduces latency when the user hits play. Lesson: Prefetching expensive requests (DRM keys, initial segments) while the user is idle yields a perceivable speedup.
- **Capability Negotiation:** The client tells the server exactly what it can handle (video vs game tiles, ad formats, etc.) ¹⁵. This allows the server to evolve new content types without breaking older clients. Lesson: In distributed UIs, a negotiation protocol for capabilities (rather than versioning the app) simplifies rolling out new features.
- **Hover Previews / Batched Requests:** Netflix uses a clever prefetch on hover: a single `MiniModalQuery` fetches the data for all titles in a row ¹¹⁰. When the user hovers one title, data for its neighbours is already loaded. Lesson: Batch-fetching likely-needed data (e.g. an entire carousel) amortises requests and reduces future latency.
- **Video URLs and CDN Failover:** The OCA URLs contain region/IX hints (e.g. `c345-sjc002-ix`) but actually resolve to anycast. Our capture showed the player connected to **three different**

OCA nodes (`c345` , `c310` , `c314`) at the same San Jose IX during a single session ⁹⁵ . The client seamlessly switched nodes, suggesting a custom load-balancing or session recovery at the OCA level. Lesson: Redundancy at the edge is crucial; multiple servers can serve the same stream and the client can reconnect without user impact.

- **Granular A/B Flags:** Many URL query flags (`hasVideoMerchInBob` , `hasVideoMerchInJaw` , etc.) control UI features ³³ . Netflix can turn features on/off per session without code deploys. Lesson: Lightweight feature flags baked into API requests allow dynamic UI control.
- **Rich Content Model:** Netflix's GraphQL model is notably rich and connected. For example, `similar` is an array of related video IDs ¹¹¹ , enabling "More Like This" rows. Playlists actions (`ADD_TO_PLAYLIST`) are embedded in each title's data ¹¹² . Lesson: A unified graph model (as enabled by GraphQL) simplifies UI logic, since all needed fields come in one query.
- **Traceability:** Every response had a `x-b3-traceid` header ⁴⁵ . This means Netflix has end-to-end tracing with B3 (Zipkin style). Lesson: Instrumenting all services with trace IDs greatly aids debugging in complex systems.
- **No Service Worker on Web:** Interestingly, Netflix does *not* use a browser service worker for offline caching ¹¹³ . They rely on native app support. Lesson: Even for a SPA, a strategic choice may be to avoid caching complexity on web, focusing instead on native apps for offline.

Each of the above points is supported by either direct captures or documented sources. For example, the hover-preview strategy is documented in the payload analysis ¹¹⁰ . The shuffle of OCA hosts during playback is noted in the teardown's non-obvious details ⁹⁵ . These serve as real-world case studies in high-scale engineering.

References

- Tandon, N., *Netflix content distribution through Open Connect*, APNIC Blog (2018) ¹ ¹¹⁴ .
- Netflix Open Connect website and documentation ¹¹⁵ ² .
- Wallace, G., *Open Connect Case Study*, FreeBSD Journal (Jan/Feb 2021) ² ⁷⁶ .
- Kleppmann, M., *Designing Data-Intensive Applications*, Ch.9 (for CAP theorem context).
- System Design Primer (alex-lvs/system-design-primer) – architecture patterns (for general comparison).
- "Netflix's web architecture reverse-engineered" (teardown gist) ¹¹⁶ ³ .
- Netflix TechBlog posts on Falcor/GraphQL migration (e.g. *Migrating Netflix to GraphQL Safely*).
- *Microservices - Netflix TechBlog*.
- APNIC, FreeBSD sources as above (cited).
- [User Provided] GitHub gist by **ssh12** on Netflix internals ¹¹⁶ ⁵⁹ .

Appendix: Diagrams and Tools

- **Mermaid Diagrams:** The above flowcharts and sequence diagrams are expressed in [Mermaid](#) syntax. To render them, you can use the Mermaid CLI:

```
npm install -g @mermaid-js/mermaid-cli
mmdc -i architecture.mmd -o architecture.png
```

Replace `.mmd` content with the diagram code blocks above.

- **Graphviz:** For more complex architecture diagrams, Graphviz `dot` can be used. Example: `dot -Tpng arch.dot -o architecture.png`.
- **Image Placeholders:** Figures 4a/4b (Open Connect modes) were adapted from Netflix's APNIC diagrams. One can recreate similar schematics using any drawing tool or Mermaid.
- **Data Modeling:** The content schema (Video/Show) is summarised from GraphQL introspection. Tools like [graphqlviz](#) could visualise it from an introspection JSON.
- **Traffic Analysis:** The technical details were derived by inspecting HTTP captures (e.g. using browser devtools or mitmproxy). For replication, one could capture Netflix traffic via browser on an AWS server (region us-west-2) to see live endpoints.

All diagrams above are illustrative. The Mermaid code can be copy-pasted into Mermaid Live Editor or integrated into documentation build pipelines. The AWS region and endpoint addresses may vary in different Netflix deployments.

1 6 7 18 19 66 68 69 70 71 72 73 78 114 Netflix content distribution through Open Connect | APNIC Blog

<https://blog.apnic.net/2018/06/20/netflix-content-distribution-through-open-connect/>

2 67 74 75 76 77 79 freebsdoundation.org

<https://freebsdoundation.org/wp-content/uploads/2021/03/Netflix-Open.pdf>

3 4 5 9 10 11 12 13 14 15 16 17 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 80 81 82 83 84 85 86 87 88 89 90 91 92 93 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 116 Netflix's web architecture reverse-engineered from live network traffic -- 18

named internal systems (Akira, Cadmium, Shakti, Pinot, MSL, FTL, Ichnaea...), dual API migration (Falcor → GraphQL), video streaming pipeline, DRM flow, search capability negotiation, and the full content data model. All from 177 captured requests. · GitHub

<https://gist.github.com/sshh12/dda3a89514f850c459380b18b1f7eb7b>

8 94 Netflix System Design- Backend Architecture - DEV Community

<https://dev.to/gbengelebs/netflix-system-design-backend-architecture-10i3>

115 Netflix | Open Connect

<https://openconnect.netflix.com/en/>